Langchain

Llm(latge language model) – API:- openi ai , lama ….
Here llm have problems :-

- limited context window means input limit how many tokens we can pass
- outdated knowledge
- data privacy
- cost
- no connection with third party tools

its have some limited power
Gen AI Application :-
Gen ai application data input - .txt , .pdf , .word , .sql , .html …..so on
Actions – Rag , agnets , apis , manipulate , transformation , memory , 3rd party ….
Data output - image , text , video
Here open ai , lama , mistral … those all have their own frameworks
If we learn open ai frame work it can't bes used in lama we need to learn lama frame work for lama so that's impossible to learn every framework here **Langchain** comes
Alternative of lang chain is lamaindex
Why we choose langchain :- langgraph(agents) , langsmith(monitoring) . langserver(deployment) – llm tools and technology
Langchain evolution : -
Inside llm model we have **Completion model , Chat model**

- From llm model to chat model
- From legacy to LCEL (langchain expression language)
- From LC to LC Ecosystem
- From terrible documentation to ……

Generative Ai space is the fastest evolving space in software . no matter how hard I work the content will never be 100% up-to-date
Note:- but you will be ready
Ex: I don't give fish , I just learn how to fishing

Learning path :-

- Introduction
- How to connect/talk with llm ,like open ai / open source
- How to work with data - (basics of rag)
- Chains | runable
- Memory
- Eco system (langsmith , langserve , langraph)
- Level-1 application
- Level -2 application
- Level-3 application - professional full stack

Connect with leading LLMS (open ai gpt-4, gpt-40 mini, and open source models like LLAMA3 and Mistral)

Semantic Search : - understand the meaning of word and search for meaning related words
42:54
How to talk with llm with langchain :-

- Connect with the most used llm - openai
- Connect with alternatives llm – opensource
- Talk to llm with different languages
- Introduction to prompt templates
- Chains
- Output persons

Connect with llm :-

```
import os
from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv())  # read local .env file
openai_api_key = os.environ["OPENAI_API_KEY
from langchain_openai import ChatOpenAI
chatModel = ChatOpenAI(model="gpt-3.5-turbo-0125")
messages = [
    ("system","Explain about llm definition"),("human","what is the uses of llm")
]
response = chatModel.invoke(messages)
```

Connect with alternatives llm – opensource

```
import os
from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
from langchain_groq import ChatGroq
llamaChatModel = ChatGroq(model="llama-3.3-70b-versatile")
mistralChatModel = ChatGroq(model="qwen/qwen3-32b")
messages = [
    ("system","What is google?."),
    ("human","what is the uses of google."),
]

llamaResponse = llamaChatModel.invoke(messages)
```

Prompt Template For Integarting Logic Into LLM Interacttions :-
Prompt :- it is a programming language for llm to perform communication

- Zero short prompting
- Few short prompoting

```
from langchain_core.prompts import ChatPromptTemplate
chat_template = ChatPromptTemplate.from_messages([
    ("system","you are and {profession} expert {topic}"),
```

```
    ("human","hello mr {profession} can you please answer."),
    ("ai","sure!"),
    ("human","{user_inpur}")
])
```

```
messages = chat_template.format_messages(
    profession="software",
    topic="Python programming",
    user_inpur="what is the best way to learn python?"
)
response = chatModel.invoke(messages)
print(response.content)
```

Chains for sequence instructions :- it a state of execution

```
from langchain_core.prompts import FewShotChatMessagePromptTemplate
examples =[
    {"input":"hi" , "output":"hola"},
    {"input": "bye!" , "output":"adios!"}
]

example_prompt = ChatPromptTemplate.from_messages([
    ("human" , "{input}"),
    ("ai" , "{output}")
])
```

```
few_shot_prompt = FewShotChatMessagePromptTemplate(
    example_prompt=example_prompt,
    examples=examples
)
```

```
final_prompt = ChatPromptTemplate.from_messages([
    ("system" , "you are a helpful assistant that translates english to spanish"),
    few_shot_prompt,
    ("human" , "{input}"),

])
```

```
chain = final_prompt | chatModel
res = chain.invoke({"input" : "how are you?"})
print(res.content)
```

langchain expression language :-

Output parser for response formatting :-

Working with custom data (data loaders)$RAG basics concepts : -

Agenda : Data Loader - how to work with custom

Introduction to rag (retrival agumented generation)

splitter(break down the rag)
Embedding
Vector store
Retriverse
Top K

NEXT : - Rag with langchain expression language
2:39

Drawbacks in token generating in openAI:-
Input imit issue
Cost

To overcome this drawback (RAG) technique come matter :-
RAG:- Load large data assets and ask the llm Question about it
What id the loaded large data , how to use rag
Large Documents / Database
↓
Load data
↓
Chunk documents (split document in small chunks)
↓
Create embeddings(transform text in numeric)
↓
Vector Database(load embeddings to VD)
↓
User Query(load user question and retrieve the revelant embedding to respond)
↓
Query Embedding
↓
Similarity Search
↓
Top relevant chunks
↓
LLM(sent the embedding to llm to format response properly)
↓
Answer

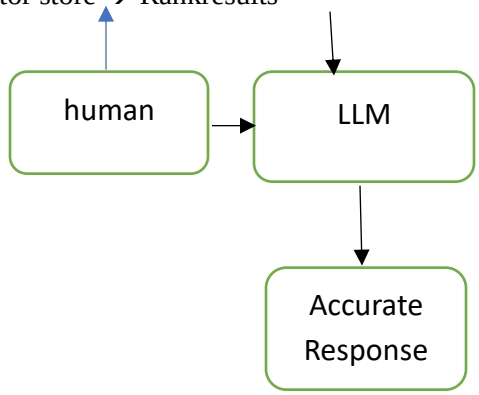
Different Rag Components like (splitters , embeddings , vector stores , retrivers , top K)
Splitters : divide large data into smaller data use **recurssive char splitter** technique and **char text splitter**
Embeddings : converting text into numbers (vectors) so that a computer can understand the semantic meaning of the text and find similar information.

Input text ⟶ embeddeding ⟶ vector data(numeric)

vector stores : store the embeddings a very fast searchable database
Vector bd - Chromadb - create separate library for vector database(local database)
Vector db - faiss - local database
Vector store as retriver :- find the embedding that best answer to your question
3:23

Basic RAG Implementation with LCEL :-
3:26

RAG Architecture :-
Custom data → spltting data → embedding → vector embedding → vector store → Rankresults



LCEL syntax:-
LCEL = LangChain Expression Language. It lets you connect LangChain components using the | pipe operator.
Basic LCEL syntax
chain = prompt | model | parser
The data flows from **left → right**:
Input
↓
prompt
↓
model
↓
parser
↓
Output
Then execute:
response = chain.invoke(input)

For user prompt we ar using **ChatPromptTemplate** it will creating the chat template
2nd generation langchain :-
StrOutputParser :- convert llm output into nrml string
RunnablePassthrough :- passes the input through unchanged

These two are libraries in 2nd generation for

3:44

Memory management in langchain Temporary and Permanent :-

Memory type	What it does
ConversationBufferMemory	Stores the entire conversation
ConversationBufferWindowMemory	Stores only the last **K** messages
ConversationSummaryMemory	Summarizes older conversation
ConversationSummaryBufferMemory	Keeps recent messages + summarizes older ones
ConversationTokenBufferMemory	Keeps conversation within a token limit
VectorStoreRetrieverMemory	Stores memories in a vector database and retrieves relevant ones

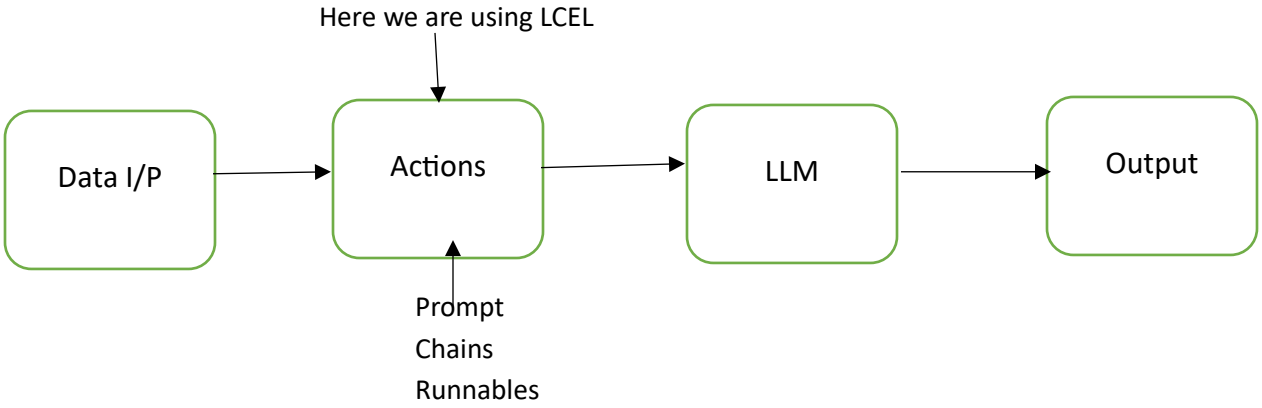
4:11:33

Introduction to langchain expression language (Lcel) chains and runnables :-

This is second generation langchain

- Runnables
- LCEL chain - sequence of runnnables
- Runnable execution order
- Runnable execution alternative
- Built-in runnable and functions
- Basic LCEL chain operation

High level architecture in GenAi



LCEL (Langchain Expression Language) : -

LCEL (LangChain Expression Language) is a way to build LangChain applications by combining components called Runnables into a chain.

It was introduced to make LangChain applications more modular, reusable, and composable.

Intro answer:-

LangChain is a framework for developing applications powered by LLMs, while LCEL (LangChain Expression Language) is a declarative syntax within LangChain used to compose Runnables into executable chains.

Think of **LangChain as a toolbox** .
Inside the toolbox you have:

- Prompt
- LLM
- Retriever
- Memory
- Tools
- Agents
- Output Parser
- Runnables

LCEL is the way you connect the tools together:
Prompt → LLM → Parser
LangChain = framework
LCEL = chain-composition mechanism within LangChain.
Runnable means (which objects we are execute those called runnable)
EX: chain = prompt | model | outputparser
The basic LCEL syntax is:
chain = prompt | llm | parser
The | operator means:
Take the output from the component on the left and pass it as input to the component on the right.
For example:
User Input
↓
Prompt
↓
LLM
↓
Output Parser
↓
Final Answer
Main goal to use LCEL :-
LCEL |
├— Composition ├— Reusability ├— Standard execution ├— Streaming ├— Parallel execution ├— Easy integration └— Building complex chains from simple components

sequence of runnnables :
chain = prompt | chatModel | output_parser
Runnable execution order in a LCEL chain
User I/P ({“question” : “india”}) → Prompt template (“Tell me about curios fact about india”) → LLMModel (“AIMessage(……..)”) → outpuparser((……..))
Runnable execution alternative

Invoke / stream / branch
4:36:40
Built in Runnables in LCEL :-
Runnable
├── **RunnableSequence**
├── **RunnableParallel → MAIN**
├── **RunnablePassthrough → MAIN**
├── **RunnableLambda → MAIN**
├── **RunnableBranch**
└── **RunnableMap**
RunnableLambda :-
RunnableLambda is used to wrap a custom Python function so that the function can be integrated into an LCEL pipeline as a Runnable.
RunnablePassthrough :-
RunnablePassthrough is used when you want to **pass the input through unchanged** to the next step in an LCEL chain.

Agenda :-

- Built in functions in runnable
- Combining chains
- Rag Demo

Bs_kwargs - Pass these arguments to BeautifulSoup
BeautifulSoup internally to parse the HTML. (parse - to break something down into its smaller parts and analyze them)
parse_only=bs4.SoupStrainer(...) This is saying –

- **Don't parse/extract the entire webpage. Only look at the HTML elements that match my conditions.**

SoupStrainer is a BeautifulSoup feature used to tell BeautifulSoup:
Only give me the HTML elements matching these conditions**.**

Load custom data → create chains → customize output from llm → connect with different llm → add memory
2nd version = introduce EcoSystem
Langchain EcoSystem :-

- Langsmith
- Langserve
- Langgraph

Crew AI
1. **LangSmith** — monitor and debug

LangSmith is used to observe, debug, test, and evaluate LLM applications.
2. **LangServe** — deploy your chain as an API
LangServe is about **serving LangChain applications**.
3. **LangGraph** — build AI agents/workflows
It is used when your application needs **multiple steps, decisions, loops, memory, or agents**.

CrewAI is a framework for building **multi-agent AI applications**.
The easiest way to understand it is:
CrewAI lets you create a team of AI agents, give each agent a role, and make them work together to complete a task.

5:37:55
Langserver :-

- It is langchain ecosystem LLM oops tool
- Build simple llm apps / webapps / api

Simple translation app :-
Language
Text

LangGraph :-

Project : -
Langchain
Openai – gpt 3.5
Vector DB – FAISS (local v DB)
Fast APi

